

Exercices 17 — Tri par insertion · Terminaison

Semaine 17 ■ À faire sans machine ■ 1 h

Exercice 1 — Dérouler le tri ★

Trier $[4, 1, 5, 2]$ par insertion : donner l'état du tableau après chaque **insertion** (étapes $i = 1, 2, 3$).

Exercice 2 — Compter les décalages ★★

Pour chaque tableau de 6 éléments, donner (sans dérouler entièrement) le nombre **total de décalages** $t[j] = t[j - 1]$ effectués par le tri par insertion :

1. $[1, 2, 3, 4, 5, 6]$
2. $[6, 5, 4, 3, 2, 1]$
3. $[2, 1, 4, 3, 6, 5]$

Exercice 3 — L'ordre des conditions ★★

Un élève a écrit :

```
while t[j - 1] > valeur and j > 0:
    ...
```

1. Sur quel déroulé ce code se comporte-t-il mal ? (Penser à une valeur plus petite que tout le début du tableau.)
2. Pourquoi Python ne lève-t-il même pas d'erreur ?

Exercice 4 — Quel tri choisir ? ★★

Pour chaque situation, choisir sélection ou insertion, et justifier :

1. Un classement de tournoi mis à jour après chaque match (une valeur change, le reste est trié).
2. Des données complètement aléatoires, triées une seule fois.
3. Un capteur ajoute chaque minute une mesure à une liste déjà triée.

Exercice 5 — Variants de boucle ★★

Pour chaque boucle, proposer un variant (entier, positif, strictement décroissant) ou expliquer pourquoi on n'en connaît pas :

```
# A
while n > 0:
    n = n - 3

# B
while a != b:
    if a > b:
        a = a - b
    else:
        b = b - a

# C
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
```

Exercice 6 — Insertion dans une liste triée ★★★

Sans trier tout le tableau : écrire `insérer(t, x)` qui insère `x` à sa place dans une liste **déjà triée** (utiliser `t.append(x)` puis faire glisser — c'est exactement une étape du tri par insertion). Quel est son coût au pire ?